Memoria técnica: LightsOut

Nombre:Nick Constante

1. Introducción

LightsOut es una aplicación Android escrita en Kotlin que implementa el juego Lights Out. El tablero es una cuadrícula de casillas que pueden estar en dos estados: gris (apagada) o morado (activada). Al pulsar una casilla cambian de estado ella misma y sus vecinas directas (arriba, abajo, izquierda y derecha). El objetivo es dejar todas las casillas en morado.
Esta memoria describe la lógica con la que se representa y recorre la matriz de 3 × 3, el algoritmo de adyacencia, el aspecto de la aplicación en funcionamiento y los componentes técnicos empleados. La versión «Mejoras» del proyecto añade, además del tablero de 3 × 3, las siguientes funciones:

- Selector de tamaño del tablero (3 × 3, 4 × 4 y 5 × 5) mediante un Spinner; la misma lógica sirve para los tres porque depende de la variable tamano.
- Contador de movimientos y cronómetro con formato mm:ss.
- Retroalimentación con sonido (ToneGenerator) y vibración (Vibrator) en cada pulsación y al ganar.
- Botón Nueva partida (tablero aleatorio que siempre tiene solución) y botón Reiniciar (vuelve al tablero con el que empezó la partida).
- Bloqueo del tablero una vez ganada la partida.

2. Diagrama de arquitectura y flujo

2.1 Arquitectura general

La aplicación tiene una única pantalla. Toda la lógica vive en MainActivity, que lee los recursos XML, registra los listeners de la interfaz y mantiene el estado del juego. La vista nunca guarda el estado: el estado lógico está en un array y la pantalla se redibuja siempre desde él con el método pintar().

Variables de estado principales

Variable	Significado
tamano	Casillas por lado (3, 4 o 5). Se calcula como posición del Spinner + 3.
estado: BooleanArray	Estado lógico de cada casilla: false = gris, true = morado.
estadoInicial	Copia del tablero con el que empezó la partida (para Reiniciar).
botones: ArrayList<Button>	Botones del tablero activo, en el mismo orden que estado.
movimientos, segundos	Contadores de la partida actual.
juegoTerminado	Pasa a true al ganar; impide seguir jugando.

2.2 Mapeo de la matriz 3 × 3

El tablero no se guarda como matriz bidimensional, sino como un array lineal de tamano × tamano posiciones recorrido por filas. Para el tablero de 3 × 3 las casillas se numeran del 0 al 8, y la lista botones guarda los botones en el mismo orden, de modo que botones[i] es siempre la vista de estado[i].



Numeración lineal de la matriz 3 × 3 y casillas que cambian al pulsar el centro y una esquina.
La conversión entre la posición lineal i y sus coordenadas se hace con división y resto enteros: fila = i / tamano y columna = i % tamano; la operación inversa es i = fila · tamano + columna. Por ejemplo, en 3 × 3 la casilla 5 está en la fila 1, columna 2.
Cada botón del layout tiene un id con el patrón t{tamano}_{i} (por ejemplo t3_5). En prepararTablero() se construye ese nombre y se obtiene el botón con resources.getIdentifier(...) y findViewById, lo que evita escribir a mano 50 búsquedas.

2.3 Algoritmo de adyacencia

El método cambiarVecinas(i) invierte la casilla pulsada y las vecinas que existan. Cada vecina solo se cambia si no se sale del tablero, comprobando la fila o la columna:

```
private fun cambiarVecinas(i: Int) {
    val fila = i / tamano
    val columna = i % tamano
    cambiar(i)                          // la pulsada
    if (fila > 0) cambiar(i - tamano)          // arriba
    if (fila < tamano - 1) cambiar(i + tamano)  // abajo
    if (columna > 0) cambiar(i - 1)            // izquierda
    if (columna < tamano - 1) cambiar(i + 1)    // derecha
}
```

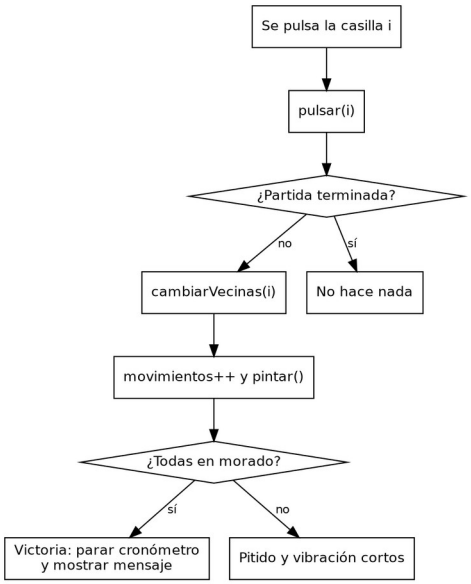
Con esto, una esquina afecta a 3 casillas, un borde a 4 y el centro a 5. El coste de una jugada es constante. Además, pulsar dos veces la misma casilla deja el tablero como estaba y el orden de las pulsaciones no importa; ambas propiedades se aprovechan al generar el tablero (apartado 2.5). Resultado para cada casilla del 3 × 3:

Casilla i	(fila, col)	Tipo	Casillas que cambian
0	(0, 0)	Esquina	0, 1, 3
1	(0, 1)	Borde	0, 1, 2, 4
2	(0, 2)	Esquina	1, 2, 5
3	(1, 0)	Borde	0, 3, 4, 6

Casilla i	(fila, col)	Tipo	Casillas que cambian
4	(1, 1)	Centro	1, 3, 4, 5, 7
5	(1, 2)	Borde	2, 4, 5, 8
6	(2, 0)	Esquina	3, 6, 7
7	(2, 1)	Borde	4, 6, 7, 8
8	(2, 2)	Esquina	5, 7, 8

2.4 Flujo de una pulsación

Cuando el jugador pulsa un botón del tablero, el listener llama a pulsar(i). El método ignora la pulsación si la partida ya ha terminado; en caso contrario aplica la jugada, actualiza el marcador, repinta el tablero y comprueba si se ha ganado.



2.5 Generación de un tablero con solución garantizada

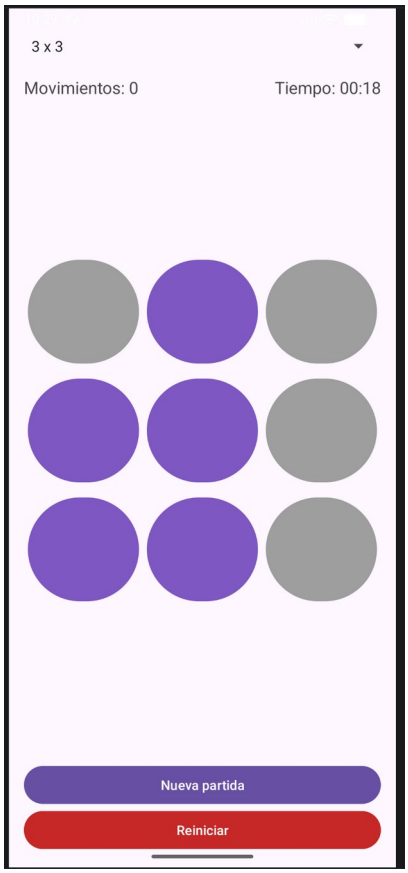
No basta con colorear casillas al azar, porque algunas configuraciones de Lights Out no tienen solución. Por eso nuevaPartida() parte del tablero ya ganado (todas en morado) y simula tamano² pulsaciones aleatorias con cambiarVecinas. Como cada pulsación se deshace repitiéndola, volver a pulsar las mismas casillas devuelve el tablero a su estado ganado: la solución existe siempre. Si por casualidad las pulsaciones se anulan y el tablero queda ya ganado, el bucle do … while (hayVictoria()) lo vuelve a generar.

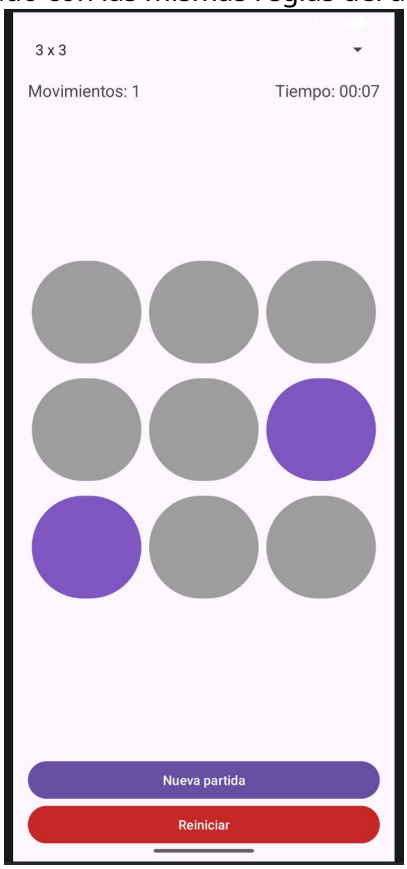
2.6 Detección de victoria

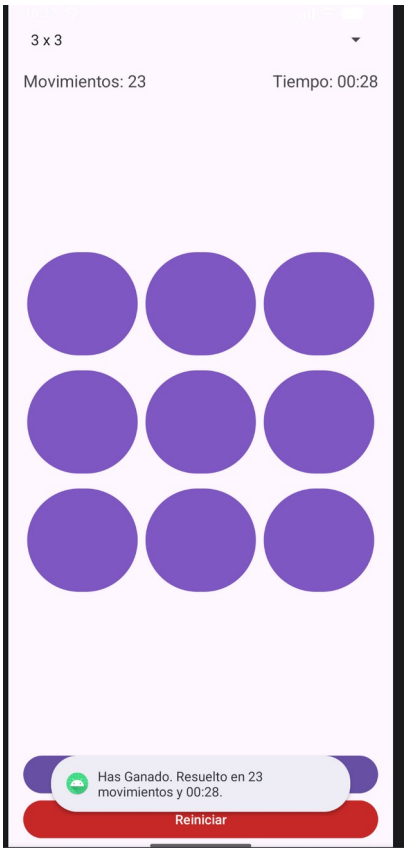
hayVictoria() recorre estado y devuelve false en cuanto encuentra una casilla en gris; si llega al final, todas están en morado. Se comprueba después de cada jugada. Al ganar se detiene el cronómetro, se activa juegoTerminado, suenan un tono largo y una vibración de 300 ms, y se muestra un Toast con los movimientos y el tiempo empleados.

3. Capturas de pantalla

Las siguientes imágenes muestran la aplicación con el tablero de 3 × 3 en tres momentos de una misma partida. Reproducen la interfaz de forma simplificada a partir del layout y los colores del proyecto (casilla apagada #9E9E9E, casilla activada #7E57C2, botón Reiniciar #C62828); los estados del tablero se han calculado con las mismas reglas del algoritmo descrito arriba.







- Estado inicial. Tablero recién generado: movimientos 0 y tiempo 00:00. Tiene cuatro casillas en gris, (0, 2, 5 y 8), y el cronómetro ya está en marcha.
- Estado intermedio. Tras pulsar la casilla 4 cambian ella y sus vecinas 1, 3, 5, 7. El marcador muestra 1 movimiento a los 7 segundos; el tablero tiene ahora 6 casillas en gris.
  (El tiempo no coincide porque para hacer foto tuve que darle a reiniciar)
- Pantalla de victoria. Tras pulsar también las casillas 4 y 8, todas las casillas están en morado. El cronómetro se detiene, el tablero queda bloqueado y aparece el mensaje «Has Ganado. Resuelto en X movimientos y X Tiempo.».

4. Detalle técnico

4.1 Detección de eventos de clic y listeners

Todos los eventos se gestionan con listeners registrados desde Kotlin en onCreate() y prepararTablero(). Se ejecutan en el hilo principal, por lo que pueden modificar la interfaz directamente.

Componente	Listener / mecanismo	Acción
Botones del tablero	setOnClickListener { pulsar(i) } (lambda)	Aplica la jugada sobre la casilla i. El valor de i queda capturado en la lambda al crear cada listener.
btnNuevaPartida	setOnClickListener { nuevaPartida() }	Genera un tablero aleatorio con solución.
btnReiniciar	setOnClickListener { reiniciar() }	Restaura estadoInicial y pone a cero los contadores.
spinnerTamano	AdapterView.OnItemSelectedListener (objeto anónimo)	tamano = position + 3, prepararTablero() y nuevaPartida(). También se dispara al abrir la app, y así empieza la primera partida.
Cronómetro	Handler(Looper.getMainLooper()) + Runnable con postDelayed(..., 1000)	Suma un segundo, actualiza el texto y se reprograma a sí mismo.

En prepararTablero() se muestra el GridLayout del tamaño elegido y los otros dos se ocultan con View.GONE. Después se vacía la lista botones, se recoge cada botón del tablero activo y se le asigna su listener, de forma que los botones de los tableros ocultos no reciben eventos ni consumen lógica.
Cada pulsación produce además retroalimentación: un pitido corto (TONE_PROP_BEEP, 80 ms) y una vibración de 30 ms; al ganar, un tono más largo (TONE_PROP_ACK, 600 ms) y una vibración de 300 ms.

4.2 Gestión de recursos de color y estilo

Los colores del juego no están escritos en el código, sino en res/values/colors.xml. En onCreate() se leen una sola vez con ContextCompat.getColor(this, R.color....), que devuelve un entero ARGB guardado en colorPrimario y colorSecundario. El método pintar() recorre estado y aplica a cada botón setBackgroundColor(...) con uno u otro color, por lo que cambiar el aspecto del juego solo requiere editar colors.xml.

Recurso	Valor	Uso
R.color.primario	#9E9E9E (gris)	Casilla apagada (estado[i] == false).
R.color.secundario	#7E57C2 (morado)	Casilla activada (estado[i] == true); el objetivo es que todas tengan este color.
backgroundTint de Reiniciar	#C62828 (rojo)	Botón Reiniciar, definido en el layout, con texto blanco #FFFFFF.

El resto del estilo procede del tema Theme.Material3.DayNight.NoActionBar (claro u oscuro según el sistema, sin barra de acción). Los textos variables usan cadenas con formato de strings.xml, p. ej. getString(R.string.time_fmt, minutos, segundos) con Tiempo: %1$02d:%2$02d. En el layout, los botones del tablero usan layout_columnWeight y layout_rowWeight iguales a 1 con margen de 4 dp, y cada GridLayout tiene layout_constraintDimensionRatio="1:1", de modo que el tablero se adapta al ancho de la pantalla y siempre es cuadrado.

4.3 Gestión de recursos y ciclo de vida

- Cronómetro: antes de arrancar uno nuevo, empezarPartida() llama a removeCallbacks(tareaCronometro), y la victoria y onDestroy() también lo hacen, para que no haya cronómetros duplicados ni tareas pendientes tras cerrar la pantalla.
- Sonido: el ToneGenerator se crea en onCreate() (volumen 80 sobre 100) y se libera con release() en onDestroy().
- Vibración: requiere el permiso android.permission.VIBRATE, declarado en el manifiesto.
- Vistas: los 50 botones se crean una sola vez en el XML; la app solo cambia su visibilidad, su color y su listener, sin crear vistas durante el juego.

5. Limitaciones y posibles mejoras

- El estado no se guarda al girar el móvil: al recrearse la Activity, el Spinner dispara onItemSelected y empieza una partida nueva. Se resolvería con onSaveInstanceState o un ViewModel.
- Vibrator.vibrate(long) está obsoleto desde Android 8 (API 26); se recomienda VibrationEffect.
- El color rojo de Reiniciar y los textos «Reiniciar» y «Has Ganado.» están escritos directamente en el layout y el código, aunque strings.xml ya tiene reset y win_title; moverlos facilitaría el mantenimiento y la traducción.

6. Conclusión

LightsOut separa con claridad el estado lógico (BooleanArray), las reglas (cambiarVecinas, hayVictoria) y la presentación (pintar, recursos XML). Representar la matriz como un array lineal con las fórmulas fila = i / tamano y columna = i % tamano permite un único algoritmo de adyacencia válido para los tableros de 3 × 3, 4 × 4 y 5 × 5, y generar los tableros a partir de pulsaciones aleatorias sobre el tablero ganado garantiza que todas las partidas tengan solución.